Error Detection: Transferring a file - Integrity Lab (SHA-256)

This lab ships a new codebase that (1) extracts the originals from **"16 Transfering a file.zip"**, (2) builds a **SHA-256 manifest** for them, (3) creates **alternate (changed) files** for every original, and (4) includes a simple **TCP file transfer** with end-to-end hashing.

Learning goals

- Detect file changes across a directory tree using cryptographic hashing (SHA-256).
- Understand manifest-based verification and **tamper detection**.
- Verify integrity across a network transfer.

What's included

- `originals/` – the extracted files from *16 Transfering a file.zip*.
- `alternates/` – per-file modified copies for testing (minimal changes that still flip hashes).
- `manifests/` – `originals_manifest.json` and `alternates_manifest.json`.
- `tools/` – `build_manifest.py`, `verify_against_manifest.py`, `transfer_server.py`, `transfer_client.py`, `util.py`.
- `received/` – output folder for the transfer server.

Requirements: Python 3.9+, no external dependencies.

Quick start

1. **Verify originals (should pass):**

```
python tools/verify_against_manifest.py --manifest
manifests/originals_manifest.json --dir originals
```

2. **Verify alternates against the originals manifest (should fail with changed files):**

```
python tools/verify_against_manifest.py --manifest
manifests/originals_manifest.json --dir alternates
```

3. **Transfer a file with verification:**

```
# Terminal A (server)
python tools/transfer_server.py --port 9100 --out received
# Terminal B (client)
python tools/transfer_client.py --host 127.0.0.1 --port 9100 --file
originals/<pick_a_file>
```

4. **Try intentional corruption:**

```
python tools/transfer_client.py --host 127.0.0.1 --port 9100 --file
originals/<pick_a_file> --inject-corrupt
```

Exercises (60–90 minutes)

A: Baseline integrity

- Inspect `manifests/originals_manifest.json` and note the algorithm and a couple of file hashes.
- Run a verification on `originals/` (expect zero changes).

Checkpoint A: Why does SHA-256 offer stronger guarantees than an additive checksum?

B: Detecting tampering

- Run a verification of `alternates/` against the **originals** manifest.
- Capture how many files are reported as **changed** vs **extra/missing**.

Checkpoint B: Explain the avalanche effect and why a 1-byte change scrambles the whole hash.

C: Make and track your own edits

- Edit one file inside `originals/` and re-verify (should now flag as changed).
- Build a new manifest and re-verify:

```
python tools/build_manifest.py --dir originals --out
manifests/my_new_manifest.json
```

Checkpoint C: When should a system **rebuild** vs **verify** a manifest? Who should own the canonical manifest?

D: Transfer with integrity check

- Start `transfer_server.py`; send a file from `originals/` with `transfer_client.py`.
- Repeat with `--inject-corrupt` and observe the server's computed hash mismatch.

Checkpoint D: Why must the server compute its **own** hash rather than trust the client's?

Stretch goals

- Zip an entire folder on the client and add a per-archive manifest; verify at the server.
- Add **HMAC-SHA-256** over header + payload for authenticity.

- Parallelize transfers and deduplicate by SHA-256 (content addressing).

Troubleshooting

- If `verify_against_manifest.py` exits with non-zero status, that indicates detected changes; this is expected for `alternates/`.
- If the server isn't reachable, ensure your firewall allows TCP port **9100**.
- Very long file names on some OSes may need path length adjustments.